Client Selection for Generalization in Accelerated Federated Learning: A Multi-Armed Bandit Approach

Dan Ben Ami, Kobi Cohen, Qing Zhao

arXiv:2303.10373v1 [cs.LG] 18 Mar 2023

***Abstract*— Federated learning (FL) is an emerging machine learning (ML) paradigm used to train models across multiple nodes (i.e., clients) holding local data sets, without explicitly exchanging the data. It has attracted a growing interest in recent years due to its advantages in terms of privacy considerations, and communication resources. In FL, selected clients train their local models and send a function of the models to the server, which consumes a random processing and transmission time. The server updates the global model and broadcasts it back to the clients. The client selection problem in FL is to schedule a subset of the clients for training and transmission at each given time so as to optimize the learning performance. In this paper, we present a novel multi-armed bandit (MAB)-based approach for client selection to minimize the training latency without harming the ability of the model to generalize, that is, to provide reliable predictions for new observations. We develop a novel algorithm to achieve this goal, dubbed Bandit Scheduling for FL (BSFL). We analyze BSFL theoretically, and show that it achieves a logarithmic regret, defined as the loss of BSFL as compared to a genie that has complete knowledge about the latency means of all clients. Furthermore, simulation results using synthetic and real datasets demonstrate that BSFL is superior to existing methods.**

***Index Terms*—Federated learning (FL), client selection, client scheduling, multi-armed bandit (MAB), generalization in machine learning.**

I. Introduction

The increasing demand for ML tasks in wireless networks consisting of a large number of clients (i.e., users or edge devices) has led to the rise of a new ML framework, called federated learning (FL) [2]–[4]. FL enables to train ML models without extracting the data directly from the edge devices. In the beginning of the FL procedure, the ML model is being initialized. Next, the FL training scheme is done by repeated iterations between the clients that implement local training and the parameter server that aggregates functions of the local trained models. This allows to train the global model without explicitly exchanging the data, which is advantageous in terms of privacy considerations and communication resources. The bandwith constraint dictates the total number of clients that can be scheduled for transmissions, which can be implemented by traditional digital communications over orthogonal channels [3]–[5] or coherent analog transmissions over multiple access channels [6]–[11]. To simplify the presentation, we focus here on traditional digital communications.

We consider an FL system with K clients sharing m orthogonal channels at each iteration (e.g., OFDMA), where $m \leq K$. Since the bandwidth is limited by m channels and the number of clients K is typically high, only a small fraction of the clients can be scheduled for transmission at each iteration [12]. The selection of a subset of m clients among K clients has been studied in the context of scheduling and spectrum access problems in traditional communication schemes (see e.g., [5], [13]–[19]). However, solving the problem in FL systems adds new challenges resulting in fundamentally different algorithms and analysis [4]. Each iteration in the FL system includes selecting a subset of m clients, distributing the model from the server to the selected clients, training locally the model by the selected clients, uploading the trained models from the clients to the server (i.e., sharing m orthogonal uplink channels using OFDMA), and finally aggregating the received models at the server to update the global model. An illustration is presented in Fig. 1. New emerging challenges in the FL

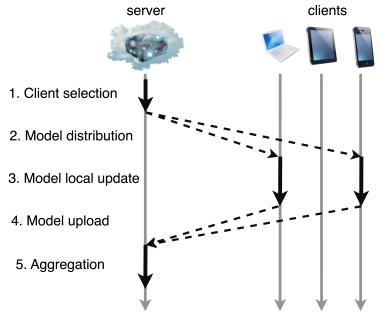


Fig. 1. An illustration of the FL communications scheme.

framework are described next [4], [20], [21]. The first challenge is the heterogeneous data between the clients [10], [21]. The local data is usually subjective to the client, and therefore is likely to be biased and imbalanced. Another challenge is balancing the total latency affected by different clients with different computational

D. Ben Ami and K. Cohen are with the School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Beer-Sheva, Israel (e-mail:danbenam@post.bgu.ac.il; yakovsec@bgu.ac.il).

Qing Zhao is with the Department of Electrical and Computer Engineering, Cornell University, Ithaca, New York, USA (e-mail: qz16@cornell.edu).

A short version of this paper was accepted for presentation in the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) 2023 [1].

This research was supported by the ISRAEL SCIENCE FOUNDATION (grant No. 2640/20)

and communication resources, as well as different channel states. This is a key challenge in FL systems since each iteration ends when the server receives the trained models from all scheduled clients, each experiences a different random latency. In this paper, we tackle these challenges and develop a novel client selection algorithm which is superior to existing methods, and achieves strong performance in terms of reducing the training latency, while not harming the generalization of the model.

A. Client Selection Methods in Federated Learning

Previous studies have shown that client selection has a great impact on the model's performance and the training latency. The standard client selection method is to choose the clients randomly with probability proportional to the amount of data each client contains [2], [22]. The clients can differ from each other in several factors such as the size of their database, the amount of available resources, their channel state, the energy they can invest, and the value of their local loss function. Each iteration in FL systems ends when the last scheduled client uploads its updated model. Thus, variability between clients might lead to poor performance of the standard random selection method. As a result, more recent methods used observations of one or more of the aforementioned factors to infer the iteration latency of each client and subsequently select the appropriate group that leads to efficient training latency [23]–[28]. In [23]–[26] the focus was on the effect of client sampling on the learning performance. In [27], [28] the problem is investigated under the assumption that parameters related to the communication link and latency are known (e.g., channel state, transmission latency). As a result, the optimization problem in those studies is deterministic with respect to the client selection strategy. However, in many scenarios, client latencies are affected by other random factors as discussed earlier (e.g., client energy state, available computational resources, random transmission rate), and are unknown at the time of the client selection. Therefore, recent studies (including this paper) tackle the problem using a fundamentally different approach by treating clients' latencies as unknown random variables drawn from unknown distributions, which leads to a stochastic optimization problem that raises the well-known exploration versus exploitation dilemma. On one hand, it is necessary to explore different actions (i.e., client selection sets) to explore the system state. On the other hand, it is imperative to exploit the information gathered so far to converge to the optimal selection strategy. This approach involves modeling and analyzing the problem as a MAB problem by treating the iteration latency or the local loss function of each client as a reward taken from an unknown distribution and given to the server (modeled as the gambler in MAB) [29]–[33]. In [29], the authors developed a naive MAB method as in the classic i.i.d. MAB that converges to a strategy that selects a small subset of the quickest clients (i.e., with the smallest expected latency). This method, however, suffers from over-fitting to the quickest clients' data when applying to scheduling in FL systems. In [30]–[32], the focus was on reducing the loss, but the generalization issue remained open. Therefore, in [33] the authors suggested to use a fairness constraint to ensure that each client is selected in a certain proportion of the communication rounds. However, this method still converges to the group of the quickest clients except for the times of selecting the slower clients due to the fairness constraint. Therefore, we suggest a novel MAB formulation for the client selection problem consisting of a reward function that captures the generalization of the ML task. This allows us to solve the problem rigorously in this paper.

B. Main Contributions

To solve the client selection problem in FL, it is needed to solve an online learning problem with the well-known exploration versus exploitation dilemma. On the one hand, the player (i.e., the server) should explore all arms (i.e., clients) in order to learn their state (i.e., latency distribution, generalization ability) which affects the overall training time of the FL task. On the other hand, it should exploit the information gathered so far to select the most rewarding subset of arms at each given time. In this paper we develop a rigorous MAB formulation to model this problem, and a novel learning algorithm to solve it. We provide rigorous theoretical analysis of the algorithm, as well as extensive numerical analysis. Below, we summarize the main contributions.

- **A novel MAB formulation for the client selection problem:** We present a new MAB formulation that trades-off between the training latency and the generalization of the model in FL by client selection. This trade-off raises new challenges in the learning design. On the one hand, it is desired to reduce the training process time of the model, and on the other hand, it is desired to increase the ability of the model to generalize by avoiding over-fitting. The novel MAB formulation tackles this trade-off by selecting clients based on a time-varying reward influenced by the history of previous selections.
- **Algorithm development:** We develop a novel upper confidence bound (UCB)-based algorithm to solve the problem, dubbed Bandit Scheduling for FL (BSFL). BSFL presents a fundamentally different approach of selecting the clients, as the reward function updates the contributions of each client to the FL task based on the history of previous selections. As a result, BSFL does not aim to converge to selecting a fixed subset of clients, but rather aims to increase the ability of the model to generalize. Furthermore, we provide concrete examples of the use of BSFL in both cases where the data is i.i.d. and balanced across clients, and where the data is non-i.i.d. and imbalanced. Since the UCB optimization in BSFL requires to solve a combinatorial problem to select the most rewarding subset of clients at

each given time, the time complexity increases exponentially with the number of clients, which might be infeasible for a large number of clients. Hence, we utilize the optimization problem's structure to devise a low-complexity algorithm, named accelerated lightweight simulated annealing (ALSA), which relies on a newly proposed accelerated lightweight simulated annealing technique that we develop for BSFL. Specifically, we show analytically that ALSA reduces the degree-order in the simulated annealing graph from $O(K^2)$ to $O(K)$ (where K is the number of clients), and still keeps the convergence property. This results in a significant accelerated convergence time requires to solve the UCB optimization in BSFL.

- **Performance analysis:** We analyze the performance of BSFL theoretically and numerically. In terms of theoretical analysis, as commonly done in the MAB literature, we evaluate the performance by regret, defined as the loss of BSFL as compared to a genie that has complete knowledge about the latency means of all clients. We analyze the performance of BSFL rigorously, and show that it achieves a logarithmic regret with time. In terms of numerical analysis, we present extensive simulations using synthetic and real datasets. All simulations demonstrate that BSFL is superior to existing methods in both the regret and the prediction accuracy.

II. System Model

We consider a common FL system with star topology. The FL system consists of a set $\mathbb{K}$ of clients with cardinality $|\mathbb{K}| = K$, where the clients communicate directly with the server via m orthogonal channels (e.g., OFDMA), where $m \leq K$. Since the bandwidth is limited by m channels and the number of clients K is typically high, only a small fraction of the clients can be scheduled for transmission at each iteration [12]. Due to operation or communication constraints, the server interacts with a set $\mathbb{A}_t \subseteq \mathbb{K}$ of clients which are available to participate in the FL task at iteration t. The number of available clients $|\mathbb{A}_t|$ is assumed to be much higher than the number of channels m. At each iteration, the server selects m clients from the set $\mathbb{A}_t$ of clients to participate in the FL task, each client is assigned to a dedicated orthogonal channel. We denote the set of all possible client selections by $H(\mathbb{K}) = \{\mathcal{S} \subset \mathbb{K} : |\mathcal{S}| = m\}$, and the set of all possible client selections at iteration t by $H(\mathbb{A}_t)$, i.e., $H(\mathbb{A}_t) = \{\mathcal{S} \subset \mathbb{A}_t : |\mathcal{S}| = m\} \subseteq H(\mathbb{K})$.

Each client $k \in \mathbb{K}$ holds a local database X_k. The local data is not being shared in FL systems due to privacy or communication constraints, and only the output model of a local training procedure is transmitted to the server. The FL task by the server is to minimize the global loss function denoted by:

$$\mathcal{L}(W, X) = \sum_{k \in \mathbb{K}} \frac{|X_k|}{|X|} \cdot \mathcal{L}(W, X_k) \tag{1}$$

with respect to the model's parameters denoted as W, where $X = \cup_{k \in \mathbb{K}} X_k$ and $\mathcal{L}(W, X_k)$ is the local loss of client k. The server's action at iteration t is defined by the selection of the participating set of clients. We denote this action in the algorithm that we aim to develop by $\mathcal{A}_t \subseteq H(\mathbb{A}_t)$. We also denote the history of all previous client selections by the server before iteration t by $\mathcal{H}_t = \{\mathcal{A}_1, \mathcal{A}_2, ..., \mathcal{A}_{t-1}\}$.

Each iteration consumes a random processing time depending on the participating clients due to distributing the model to the clients, local training and updating the model at each client, and transmitting each local model back to the server for global aggregation. We denote $\tau_{k,t}$ as the total random latency consumed by client k at iteration t. At each iteration t, the FL process proceeds to the next iteration after receiving all the trained models from all the selected clients. Therefore, the total iteration time τ_t is dictated by the slowest client. We further assume a fixed latency bound, τ_{max}, where all clients must meet so that the server receives their local model. As a result, the total latency at iteration t is given by:

$$\tau_t = \min\{\max_{k \in \mathcal{A}_t} \tau_{k,t}, \tau_{max}\}. \tag{2}$$

III. MAB-Based formulation for the client selection problem

In this section, we present a novel MAB formulation for the client selection problem in order to minimize the training latency without harming the ability of the model to generalize, i.e., to provide reliable predictions for new observations.

A. Trading-off Between the Generalization and Iteration Time via MAB Formulation

MAB problems are often illustrated with the example of a player or gambler facing a row of slot machines, selecting arms to pull at each time and obtaining rewards accordingly. In our context, the server acts as the player, and the clients act as the arms. At each iteration, the server selects a subset of clients, which contribute to the learning rate accordingly. Solving the MAB problem, and specifically the client selection problem in this paper, requires addressing the well-known exploration versus exploitation dilemma in online learning. On one hand, the server should explore all clients to learn their state, such as their latency distribution and generalization ability, which affect the overall training time of the FL task. On the other hand, the server should exploit the information gathered so far to select the most rewarding subset of clients at each given time.

The exploration versus exploitation dilemma has been rigorously addressed in MAB optimization [34]. As a result, recent studies have modeled and analyzed the client selection problem in FL as a MAB problem, treating the iteration latency or local loss function of each client as a reward obtained from an unknown distribution

and given to the server [29]–[33], as discussed in Section I-A. However, all these methods converge to selecting the fastest clients (except for the times of selecting slower clients due to the fairness constraint in [33]). Consequently, they suffer from overfitting to the data of the quickest clients when applied to scheduling in FL systems, resulting in decreased ability to generalize and decreased prediction accuracy.

In this paper, we overcome this limitation by developing a novel MAB formulation for the client selection problem consisting of a time-varying reward function that captures both the client latency as well as the client ability to contribute to the generalization of the ML task. It should be noted that previous work on rotting bandits [35] considered a reward function that decreases with the activation rate of selected arms. However, the model here is fundamentally different, since the time-varying reward function depends on the overall subset selections of correlated clients in the systems. This leads to a novel combinatorial MAB (CMAB) formulation [36], in which the design and analysis is fundamentally different from rotting bandits. Specifically, our formulation trades-off rigorously between the generalization and latency, as each client in FL systems might have different characteristics such as computing power, task operations, channel states, etc. This results in different latency distributions in training and transmitting the local models across clients in the FL system [4]. On the one hand, to reduce the total iteration time at each iteration, it is desired to select clients with small latencies at each iteration. On the other hand, it is desired to select as many clients as possible during the training process (even slower clients) for the model to be robust and generalized for both i.i.d. balanced data and non-i.i.d. imbalanced data [33], [37]–[39]. In the next susbection we describe the generalization function used in the CMAB formulation.

B. The Generalization Function

For simplicity and maintaining generality, we denote by Θ all system constant parameters that contribute to the reward (which will be described later with examples). The level of contribution of client k to the generalization ability and robustness of the model is evaluated by a generalization function $g_{k,\Theta} : \mathcal{H}_t \rightarrow [-1, 1]$. It gives at each iteration t a value that represents the contribution of client k to the generalization of the global model, depending on the model constant parameters Θ and the selection history before time t, $\mathcal{H}_t$. We next present concrete examples of the function $g_{k,\Theta}$ in both cases of i.i.d. and balanced data, and non-i.i.d. and imbalanced data across clients.

We start by considering the case of i.i.d. balanced data. We use the selection history to extract a counter $c_{k,t}$ for each client that counts the number of iterations that client k was selected until the beginning of iteration t, i.e., $c_{k,t} = \sum_{i=1}^{t-1} \mathbb{1}_{\{k \in \mathcal{A}_i\}}$. In order to achieve high generalization in this case, it is important that each client is selected an equal number of times [22], [33], [37]–[39]. In this case, we let Θ store the number of clients and the number of channels, i.e., $\Theta = \{K, m\}$. Then, an effective design of the function $g_{k,\Theta}$ is given by:

$$g_{k,\Theta}(\mathcal{H}_t) = g_{k,\Theta}\left(\frac{c_{k,t}}{t}\right) = \left|\frac{m}{K} - \frac{c_{k,t}}{t}\right|^{\beta} \cdot sgn\left(\frac{m}{K} - \frac{c_{k,t}}{t}\right), \tag{3}$$

where β is a tuning parameter (natural number), and $sgn(\cdot)$ is the sign function that returns ± 1. An illustration is presented in Fig. **??**.

The key idea behind the design is that the more the ML model is trained with new data points or alternatively with data that was used too little, the more its ability to generalize increases [37], [39]. It is thus desired to select clients uniformly with rate m/K over time. Therefore, the generalization function $g_{k,\Theta}$ is designed to be monotonically decreasing so that it provides incentive (i.e., positive value in terms of contributing to generalization) to select clients that were selected too little (i.e., $\frac{c_{k,t}}{t} < m/K$), and returns zero when the client's counter reaches the desired selection rate (i.e., $\frac{c_{k,t}}{t} = m/K$). Clients that were selected too frequently are given negative values in terms of contributing to generalization.

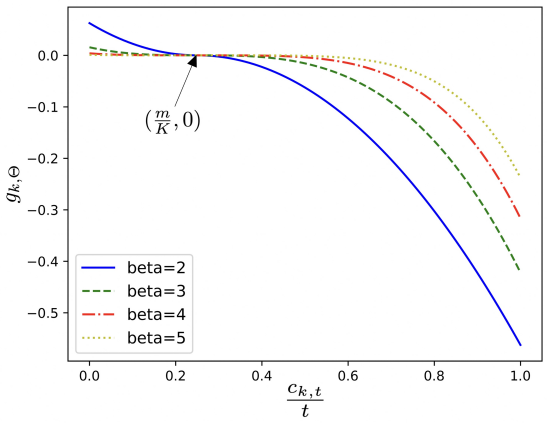


Fig. 2. The value of $g_{k,\Theta}$ as a function of $\frac{c_{k,t}}{t}$ with different values of β.

In the second scenario, consider non-i.i.d. and imbalanced data, where the data quality varies between clients. In this case, it is desired to take into account the participation rate of each client, as well as the quantity and quality of the data, when selecting clients for transmission. The quantity of each client's data (i.e., its database size $|X_k|$) is useful for the generalization, as it more likely to represent the true distribution of data [40]. The second aspect stems from the desire to train the model on reliable, high-quality, and meaningful data [41]–[44]. Generally, in ML, and particularly FL, the data for each client might have different quality due to several factors such as device heterogeneity, environment heterogeneity, etc. We define the client's data quality by $q_k \in [0, 1]$, where 1 stands for the best quality, and thus

the generalization function $g_{k,\Theta}$ is designed to prioritize clients with higher data quality. Since the quality for each client (say client k) is aggregated over all its data points X_k, we denote the overall significance of client k by $d_k = q_k \cdot |X_k|$. Therefore. the model parameters in this case is given by $\Theta = \big\{K, m, \{(|X_k|, q_k) : k \in \mathbb{K}\}\big\}$, and the generalization function is given by:

$$g_{k,\Theta}(c_{k,t}) = \left| \frac{md_k}{\sum_{k\in\mathbb{K}} d_k} - \frac{c_{k,t}}{t} \right|^{\beta} \cdot sgn\left(\frac{md_k}{\sum_{k\in\mathbb{K}} d_k} - \frac{c_{k,t}}{t} \right). \tag{4}$$

In the next subsection we present the CMAB formulation for the client selection problem. In the empirical study in this paper, we used the forms of $g_{k,\Theta}$ as detailed above, which demonstrated very good performance. It should be noted, however, that the CMAB formulation is general. It does not depend on a specific form of $g_{k,\Theta}$, and other forms can be used. For example, it is possible to define $g_{k,\Theta}$ to be dependent on the local loss value of each client, which is advantageous in improving the convergence in the long term [45].

C. CMAB-Based Formulation with History-Dependent Reward

We now formulate the client selection problem as a CMAB problem with history-dependent reward. At each iteration t, the server (i.e., the player) selects a subset of clients $\mathcal{A}_t \in H(\mathbb{A}_t)$ (i.e., arms), and at the end of the iteration observes the clients' iteration latencies. The reward given at the end of iteration t is defined by:

$$\begin{aligned} r(t) &= \frac{\frac{\tau_{min}}{\tau_{max}}}{\max_{k\in\mathcal{A}_t} \frac{\tau_{k,t}}{\tau_{max}}} + \alpha \cdot \frac{1}{m} \sum_{k\in\mathcal{A}_t} g_{k,\Theta}(\mathcal{H}_t) \\ &= \min_{k\in\mathcal{A}_t} \{\frac{\tau_{min}}{\tau_{k,t}}\} + \frac{\alpha}{m} \sum_{k\in\mathcal{A}_t} g_{k,\Theta}(\mathcal{H}_t), \end{aligned} \tag{5}$$

where α is a hyper-parameter of the generalization, balancing between the amount of the generalization and the iteration time, τ_{min} is the minimum iteration latency possible, which is the time needed to send and receive the model, as if the client required zero training time, and τ_{max} is the fixed latency bound as defined before (2). For purposes of analysis, we further assume that the reward given at the end of each iteration is quantized and the smallest difference between two different rewards is denoted by Δ_{min}, as commonly assumed in analyzing MAB-based problems. Since the iteration time is determined by the maximal latency under the selected subset of clients $\mathcal{A}_t$, the first term on the RHS of (5) represents the reward due to the iteration time, and the second term represents the reward due to the generalization. The smaller the value of α, the higher priority to select faster clients. On the one hand, this reduces the iteration time. On the other hand, it tends to select the same faster clients at each iteration, which reduces the generalization, and increases the total number of iterations needed to achieve reliable predictions.

An important insight from the CMAB formulation is that if a slow client is given high priority based on the reward function and is selected for transmission at the current iteration, then since the iteration time is determined by the slowest client, there is no motivation to choose fast clients at the same iteration. Instead, clients with high rewards in terms of generalization, even if they are slow, should be prioritized.

Note that the formulation of our CMAB problem is fundamentally different from the classical CMAB problem. Specifically, here the reward does not depend on the new observations solely (as in classic CMAB), but also on the selection history up to the current iteration. Thus, optimizing the reward in classic MAB leads to a fixed selection of arms, while in our problem the optimal selection is time-varying depending on the selection history, as will be detailed in the next subsection.

D. The Objective

We aim to find the best selection policy $\Pi = \{\mathcal{A}_1, \mathcal{A}_2, ...\}$ (where $\mathcal{A}_t$ stands for the selection at iteration t) which minimizes the training process time while preserving the ability of the model to perform the generalization. The performance of online learning algorithms are commonly evaluated by the regret, defined as the loss of an algorithm as compared to genie with side information on the system. Here, to measure the performance of our proposed policy we define the regret as follows. We denote the maximal mean reward at iteration t that can be obtained by genie that has complete knowledge about the latency means of all clients by $r^*(t)$, where genie's selection at time t is given by:

$$\mathcal{G}_t = \underset{\mathcal{S}\in H(\mathbb{A})}{\arg\max} \Big\{ \min_{k\in\mathcal{S}} \mu_k + \frac{\alpha}{m} \sum_{k\in\mathcal{S}} g_{k,\Theta}\big(\mathcal{H}_t\big) \Big\}, \tag{6}$$

where μ_k stands for the speed mean of client k (i.e., $\mu_k = \mathbb{E}\big[\frac{\tau_{min}}{\tau_k}\big]$).

The regret of the policy (say Π) at time n is defined by the accumulated loss by time n between the reward obtained by genie and the expected reward obtained by Π:

$$R(n) = \mathbb{E}_{\Pi}\left[\sum_{t=1}^{n} r^*(t) - r(t) \right]. \tag{7}$$

The objective is thus to find a policy Π that minimizes the regret order with time.

IV. The Proposed BSFL Algorithm

In this section we start by presenting the BSFL algorithm to solve the objective. Then, we analyze the regret (7) analytically.

A. Description of the Algorithm

The BSFL algorithm is based on a novel UCB-type design for client selection in FL. The pseudocode for the BSFL algorithm is provided in Algorithm 1. We now discuss the steps of the BSFL algorithm in detail.

BSFL observes realizations of the random speed of the selected clients (i.e., $\frac{\tau_{min}}{\tau_{k,t}}$) and estimates its mean accordingly. Let

$$\overline{\mu_{k,t}} = \frac{1}{c_{k,t}} \sum_{i=1}^{t-1} \frac{\tau_{min}}{\tau_{k,i}} \cdot \mathbb{1}_{\{k \in \mathcal{A}_i\}} \tag{8}$$

be the sample-mean speed of client k after t iterations. We design the UCB function of client k's speed after t iterations by:

$$\text{ucb}(k,t) = \overline{\mu_{k,t}} + \sqrt{\frac{(m+1)\ln t}{c_{k,t}}}. \tag{9}$$

To maintain an updated $\overline{\mu_{k,t}}$ and $\text{ucb}(k,t)$ for each client, each time a client is selected to participate in the FL iteration, the algorithm observes its speed $\frac{\tau_{min}}{\tau_{k,t}}$ and updates its counter and the speed's sample-mean as follows:

$$c_{k,t} \leftarrow c_{k,t-1} + 1, \tag{10}$$

$$\overline{\mu_{k,t}} \leftarrow \frac{\overline{\mu_{k,t-1}} \cdot c_{k,t-1} + \frac{\tau_{min}}{\tau_{k,t}}}{c_{k,t}}. \tag{11}$$

Then, for each client $k \in \mathbb{K}$, the UCB function is updated as follows:

$$\text{ucb}(k,t) \leftarrow \overline{\mu_{k,t}} + \sqrt{\frac{(m+1)\ln t}{c_{k,t}}}. \tag{12}$$

At the initialization step, for each client $k \in \mathbb{K}$, $c_{k,0}$ and $\overline{\mu_{k,0}}$ are set to 0, and $\text{ucb}(k,0)$ is set to infinity. Later, in the main loop, the algorithm selects clients at each iteration according to:

$$\mathcal{A}_t = \underset{\mathcal{S} \in H(\mathbb{A})}{\arg\max} \left\{ \min_{k \in \mathcal{S}} \text{ucb}(k, t-1) + \frac{\alpha}{m} \sum_{k \in \mathcal{S}} g_{k,\Theta}\big(\mathcal{H}_t\big) \right\}. \tag{13}$$

Afterwards, for each client $k \in \mathcal{A}_t$, the algorithm observes $\tau_{k,t}$ and updates $c_{k,t}$, $\overline{\mu_{k,t}}$ using (10), (11). At the end of every iteration, $g_{k,\Theta}(\mathcal{H}_{t+1})$ and $\text{ucb}(k,t)$ for all client $k \in \mathbb{K}$ are updated using (12). Note that two different trade-off mechanisms are manifested during the algorithms. The first is between exploration and exploitation of the client latencies, while the second is between the iteration latency and the generalization ability.

B. Regret Analysis

In this subsection, we analyze the regret (7) achieved by BSFL analytically, and show that it has a logarithmic order with time. Let Δ_{max} be the maximum possible difference between the expected reward obtained by genie and by any selection $\mathcal{S}$.

Algorithm 1 BSFL Algorithm

Input: Set of client indices $\mathbb{K}$
Initialization:
1: $\forall k \in \mathbb{K} : c_{k,0} \leftarrow 0, \text{ucb}(k,0) \leftarrow \infty, \overline{\mu_{k,0}} \leftarrow 0$
Main loop:
2: **for** iterations $t = 1, 2, \ldots$ **do**:
3: Select a set of m clients using (13) (ties are broken arbitrarily)
4: Execute FL iteration
5: For each client $k \in \mathcal{A}_t$ observe $\tau_{k,t}$ and update $c_{k,t}$, $\overline{\mu_{k,t}}$ using (10), (11).
6: For each client $k \in \mathbb{K}$ update $g_{k,\Theta}(\mathcal{H}_{t+1})$ accordingly, and $\text{ucb}(k,t)$ using (12)
7: **until convergence**

To evaluate the regret of BSFL, we define $r(\mathcal{S}, \mathcal{H}_t)$ as the reward that could have been obtained at the end of iteration t, given the selection history $\mathcal{H}_t$, if selection $\mathcal{S}$ had been made. Let Δ_{max} be the maximum difference between the expected reward obtained by $\mathcal{G}_t$ and by any selection $\mathcal{S}$, given any selection history, i.e.,

$$\Delta_{max} = \max_{\mathcal{S} \in H(\mathbb{K}), \mathcal{H}_t \subset H(\mathbb{K}), t \in \mathbb{N}} \mathbb{E}\big[r(\mathcal{G}_t, \mathcal{H}_t) - r(\mathcal{S}, \mathcal{H}_t)\big]. \tag{14}$$

Note that Δ_{max} is bounded by:

$$\Delta_{max} \leq 2\alpha + \mu_{max} - \mu_{min},$$

where μ_{max} and μ_{min} are the expected speeds of the fastest and slowest clients, respectively.

Theorem 1. *At each iteration n, the regret of BSFL is upper bounded by:*

$$\Delta_{max} \cdot K \cdot \left(\frac{4(m+1)\ln n}{\Delta_{min}^2} + 1 + \frac{\pi^2}{3} \right). \tag{15}$$

The proof can be found in Appendix A.

Theorem 1 implies that BSFL achieves a logarithmic regret order with time $O(\ln n)$.

It is worth noting that the selection policy of Algorithm 1, i.e., finding the maximum value in the update rule stated in (13), can be computationally infeasible due to its exponential complexity. Therefore, in the following section, we will develop a practical solution to solve the maximization problem.

V. Complexity Reduction Using A Novel Accelerated Lightweight Simulated Annealing

The most computationally challenging aspect of the BSFL algorithm is selecting clients at the beginning of each iteration, as indicated by line 3 of the pseudocode. Specifically, the server needs to find the client selection that maximizes the expression in the update rule (13). Classic deterministic methods for finding this selection have a time complexity of $O\binom{|\mathbb{A}_t|}{m}$, which is computationally infeasible when the number of channels (m) is large. Therefore, heuristic methods are often used to find good approximate solutions to optimization problems in a finite amount of time. In particular, simulated annealing (SA) is a heuristic optimization algorithm that has strong theoretical properties of convergence guarantee as the

number of time steps increases. In the subsequent sections, we present a new SA-type algorithm that leverages the unique structure of the MAB-based FL scheduling problem to accelerate the stochastic optimization process through the design of a lightweight search space. The proposed SA-type method demonstrates significantly faster convergence compared to the classical method while still preserving the strong theoretical convergence guarantee property as the number of time steps used to execute line 3 of the pseudocode increases.

A. The Stochastic Optimization Flow

To implement SA-type algorithm to solve (13), we need to define a multi-state environment, in which each state s has neighboring states denoted as N_s. Each state also has an associated energy, represented by $E(s)$. The objective of the algorithm is to find the state with the highest energy (or, alternatively, the lowest energy, depending on the problem formulation). To facilitate movement between states, a temperature parameter T_i for time step i is introduced. This temperature affects the probability of transitioning from the current state to a state with lower energy. The algorithm begins by randomly selecting an initial state s_0. It then proceeds through a series of time steps, during which the temperature is updated and a neighboring state $u \in N_{s_i}$ is chosen at random. The next state s_{i+1} is then updated according to:

$$s_{i+1} \leftarrow \begin{cases} u, & \text{if } E(u) \geq E(s_i), \\ u, & \text{w.p., } e^{\frac{E(u)-E(s_i)}{T_i}} \text{ if } E(u) < E(s_i), \\ s_i, & \text{otherwise.} \end{cases} \tag{16}$$

In our MAB-based FL scheduling setting, each client selection determines a state, which means that in each FL round t, the number of states of the stochastic optimization problem would be $\binom{|\mathbb{A}_t|}{m}$. The energy of each state would be:

$$E(\mathcal{S}) = \min_{k \in \mathcal{S}} \left\{ \text{ucb}(k,t) + \frac{\alpha}{m} \sum_{k \in \mathcal{S}} g_{k,\Theta}\big(\mathcal{H}_t\big) \right\}. \tag{17}$$

As previously mentioned, the SA dynamics requires that each state has neighboring states. A direct implementation of the classic SA method results in a structure where any two client selections (which represent two states), $\mathcal{S}$ and $\mathcal{U}$, will be considered neighboring selections only if they differ in only one client. Formally,

$$\mathcal{S} \in N_{\mathcal{U}} \ , \ \mathcal{U} \in N_{\mathcal{S}} \Leftrightarrow \big|\mathcal{S} \cap \mathcal{U}\big| = m - 1, \tag{18}$$

where m is the number of channels, same as before. It can be seen that through this construction, the neighborhood is symmetrical between the selections (states) and that for each selection there are $m(K-m)$ neighbors. From [46], setting the temperature at each time step to $T_i = \frac{\Delta_{max}}{\log(i+1)}$ guarantees convergence to the selection with the maximum energy as the number of time steps in the SA algorithm approaches infinity, where Δ_{max} is defined in (14). Despite the strong theoretical convergence guarantee, a recognized disadvantage of the SA method is its rate of convergence which can be quite slow. Thus, in the subsequent section, we present a novel SA-type algorithm that addresses this issue while still maintaining the theoretical convergence guarantee.

B. The Proposed Accelerated Lightweight Simulated Annealing (ALSA) algorithm

Building on the concept of SA-type dynamics, we present a novel accelerated SA-type method that capitalizes on the specific structure of the MAB-based FL scheduling problem. Our proposed method accelerates the optimization process by designing a lightweight search space, resulting in faster convergence compared to the classic SA technique. This is achieved by taking into account the unique features of the MAB-based FL scheduling problem.

When applying the classic SA method to our problem, the number of neighbors for each state is $m(K-m)$, which becomes $O(K^2)$ when $m = O(K)$. In our proposed ALSA method, we exploit the characteristics of the multi-armed bandit optimization to reduce the number of connections between states and create a lightweight state graph for the search space. This modification allows for faster convergence while still maintaining the theoretical convergence guarantee of the SA method (which will be shown later). Specifically, we define a new neighborhood rule that only allows for client selections that differ by a single client from the current selection, and requires that this client has the lowest ucb value or the lowest $g_{k,\Theta}$ value in one of the selections. Formally,

$$\begin{aligned} &\mathcal{S} \in N_{\mathcal{U}} \ , \ \mathcal{U} \in N_{\mathcal{S}} \\ &\quad \Leftrightarrow \Big(\big|\mathcal{S} \cap \mathcal{U}\big| = m - 1 \Big) \text{ and} \\ &\Big(\mathcal{S} \backslash \mathcal{U} \subset \Big\{ \arg\min_{k \in \mathcal{S}} \text{ucb}(k,t), \arg\min_{k \in \mathcal{S}} g_{k,\Theta}(\mathcal{H}_t) \Big\} \text{ or} \\ &\mathcal{U} \backslash \mathcal{S} \subset \Big\{ \arg\min_{k \in \mathcal{U}} \text{ucb}(k,t), \arg\min_{k \in \mathcal{U}} g_{k,\Theta}(\mathcal{H}_t) \Big\} \Big). \end{aligned} \tag{19}$$

It is important to note that each selection $\mathcal{S}$ has at most $2(K-m)$ other selections that differ by only one client k which is in the set

$$\Big\{ \arg\min_{k \in \mathcal{S}} \text{ucb}(k,t), \arg\min_{k \in \mathcal{S}} g_{k,\Theta}(\mathcal{H}_t) \Big\}.$$

The neighborhoods in this formulation are symmetrical, and the average number of neighbors for each selection is no more than $4(K-m)$, resulting in a total of $O(K)$ neighbors regardless of the value of m.

Denote the state with the global maximum energy as s^*. The theoretical convergence of ALSA is shown next.

Theorem 2. *Implementing ALSA with temperature $T_i = \frac{\Delta_{max}}{\log(i+1)}$ at time step i yields:*

$$\lim_{i \to \infty} E(s_i) = E(s^*). \tag{20}$$

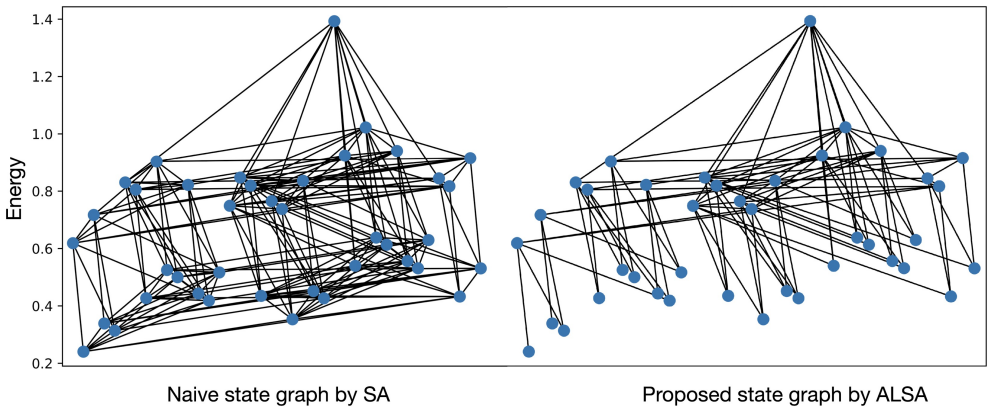


Fig. 3. An illustration of the sub-graphs of states resulting by ALSA and SA. For clarity, only sub-graphs of the complete graphs are shown.

The proof can be found in Appendix B.

Theorem 2 implies that as the number of time steps for running ALSA increases, the result of executing ALSA will converge to the optimal solution of (13). By using ALSA, the execution of line 3 in the BSFL algorithm (Algorithm 1) becomes efficient.

C. A Comparison of SA and ALSA for solving (13)

The classic SA method, when applied to our problem, results in a graph structure in which each state has $O(K^2)$ neighbors. However, by exploiting the characteristics of the MAB-based optimization, our proposed ALSA method is able to reduce the number of connections between states to create a lightweight graph for the search space, while still maintaining the theoretical convergence guarantee as previously demonstrated. This construction results in a graph structure in which each state has $O(K)$ neighbors.

To demonstrate the effectiveness of our proposed method, we conducted thousands of runs with varying numbers of clients and selection sizes. In the vast majority of runs (98.3%), ALSA reached a state with a higher energy within the fixed time period compared to the classic SA method. Furthermore, we observed that the majority of the additional edges in the classic SA method are between low-energy states or states with similar energy, which do not contribute to the convergence to the global maximum state and instead cause the algorithm to wander for a longer time between these low-energy states. This is illustrated in Figure 3, which shows a comparison between the graph structures created by ALSA and SA for a simulation of selecting 4 clients out of 8 with drawn $\text{ucb}(k,t)$ and g_k values. As can be observed in Fig. 4, the judicious removal of excess edges in the graph created by ALSA leads to a significantly faster rate of convergence to high-energy states compared to SA.

VI. Simulation Results

In this section, we present the results of our simulations, which include two types of datasets and models.

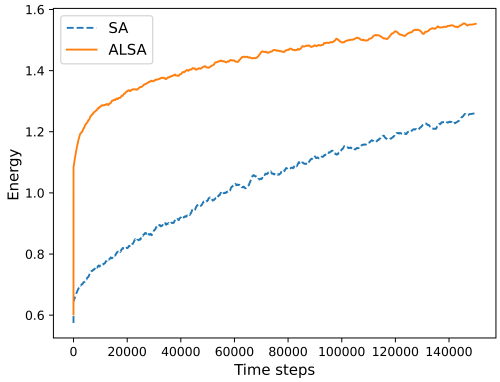


Fig. 4. The energy of the current state as a function of the time step of ALSA and SA runs is depicted in this figure. The simulation in this figure is based on the selection of 25 clients out of 500, for which it is computationally infeasible to directly solve (3) ($\approx 2^{139}$ different selections).

The first is synthetic data for linear regression and the second is image data from two well-known datasets, Fashion-MNIST and CIFAR-10, for a convolutional neural network (CNN) model.

We consider two scenarios for each simulation. In the first scenario, the data (images or synthetic data) is divided i.i.d. between the clients, and for the image databases each client has an equal number of images from each class. In the second and more challenging scenario, each client has a different amount of data and, in the image datasets, a different distribution of images among the ten classes.

In the i.i.d. case, we compared the proposed BSFL client selection algorithm to the client selection UCB (CS-UCB) algorithm proposed in [29] and to a random selection policy [2]. During each simulation, the model was trained for the same amount of time, but the different algorithms resulted in varying numbers of FL rounds (i.e., iterations) for the same amount of total training time. In other words, the number of iterations performed by each algorithm for a given latency in the figures is not the same

since the running time of each iteration varies between different client-set selections.

To compare the regret between the algorithms, we first had to search through all $\binom{K}{m}$ client selections in each iteration and determine which selection truly maximized (6). Therefore, in order to compare the regret, we chose a relatively small number of clients (namely, 20) and a selection size of 5. As shown in Figure 5 (left), BSFL achieves a logarithmic regret compared to the other algorithms, which reached a linear regret. Additionally, Figure 5 (right) illustrates that BSFL achieves the smallest loss values among the algorithms.

We start by comparing the regret between the algorithms. To compare the regret, we first had to search through all $\binom{K}{m}$ client selections in each iteration and determine which selection truly maximized (6), i.e., find the selection $\mathcal{G}_t$. Therefore, in order to compare the regret, we chose a relatively small number of clients (namely, 20) and a selection size of 5. As shown in Figure 5 (left), BSFL achieves a logarithmic regret compared to the other algorithms, which reached a linear regret. Additionally, Figure 5 (right) illustrates that BSFL achieves the smallest loss values among the algorithms.

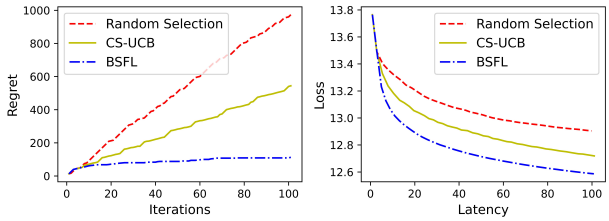


Fig. 5. Linear regression model with synthetic data in the i.i.d. scenario. Figure (left): Regret as a function of iterations. Figure (right): Test loss as a function of latency.

Second, while still operating within the i.i.d. case, we conducted simulations on real data using a CNN model with 500 clients and a selection size of 25. The CNN model employed a standard architecture comprising convolutional layers, max-pooling layers, dense layers, and dropout regularization. The activation function used in the layers was 'ReLU', with the exception of the final layer, where we used the softmax function. Additionally, we employed the categorical cross-entropy loss function for both models. In Figures 6, 7, we present the results of our simulations for each of the algorithms. The simulation results demonstrate the performance of the global model on test data throughout the training process. The figures show that BSFL significantly outperforms the other algorithms in terms of both loss and accuracy percentage on the test data.

In the non-i.i.d. scenario, we compare the proposed BSFL algorithm to the CS-UCB-Q proposed in [29] for non-i.i.d. data, and to an adapted random selection policy for non-i.i.d. data that randomly selects the clients with a probability proportional to the size of each client's database [22]. As before, to evaluate the regret, we divided the dataset into a small number of clients, and

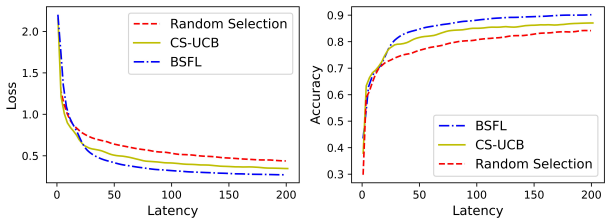


Fig. 6. CNN model with Fashion-MNIST data in the i.i.d. scenario. Figure (left): Test loss as a function of latency. Figure (right): Test accuracy as a function of latency.

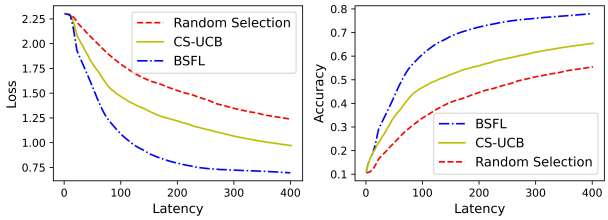


Fig. 7. CNN model with CIFAR10 data in the i.i.d. scenario. Figure (left): Test loss as a function of latency. Figure (right): Test accuracy as a function of latency.

compared the regret of each algorithm. As shown in Figure 8 (left), even in the non-i.i.d. scenario, BSFL achieves a logarithmic regret order, in contrast to the other algorithms which achieve a linear regret order. Furthermore, Figure 8 (right) illustrates how these results in terms of regret lead to faster convergence in terms of the loss calculated on the test data, indicating superior generalization of the model trained using BSFL.

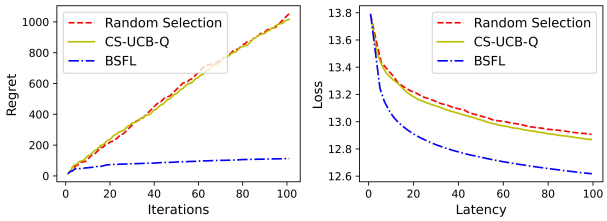


Fig. 8. Linear regression model with synthetic data in the non-i.i.d. scenario. Figure (left): Regret as a function of iterations. Figure (right): Test loss as a function of latency.

We conducted a similar experiment using real datasets and divided the data into a larger set of 500 clients, with a selection size of 25 clients per iteration. Each client contained a different number of images and varying amounts of images from each class. As shown in Figures 9 and 10, in this scenario as well, BSFL leads to faster convergence in terms of both loss and accuracy on the test data. All evaluations of the global model's performance, depicted in graphs, were conducted using test data that was not utilized for training and was not accessible to any of the clients. Therefore, we can conclude that BSFL leads to superior generalization compared to the other algorithms.

These simulation results demonstrate the strong performance of BSFL for client selection in federated learning

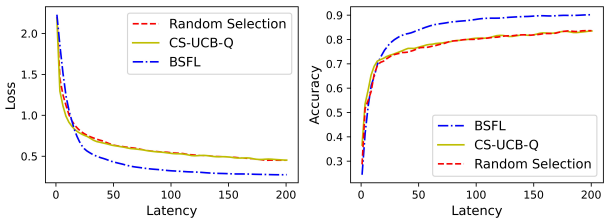


Fig. 9. CNN model with Fashion-MNIST data in the non-i.i.d. scenario. Figure (left): Test loss as a function of latency. Figure (right): Test accuracy as a function of latency.

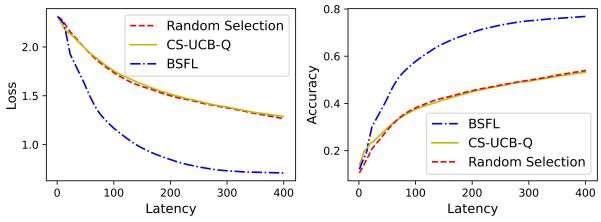


Fig. 10. CNN model with CIFAR10 data in the non-i.i.d. scenario. Figure (left): Test loss as a function of latency. Figure (right): Test accuracy as a function of latency.

compared to existing methods.

VII. Conclusion

We developed a novel MAB-based approach for client selection in FL systems, aimed at minimizing training latency while preserving the model's ability to generalize. We developed a novel algorithm to achieve this goal, dubbed Bandit Scheduling for FL (BSFL). BSFL was shown to achieve a logarithmic regret, defined as the difference in loss between BSFL and a genie with complete knowledge of all clients' latency means. Simulation results demonstrated that BSFL is superior to existing methods.

Appendix A
Proof of Theorem 1

In this appendix we provide the proof for Theorem 1. The regret of BSFL (7) can be written as:

$$\begin{aligned} R(n) &= \mathbb{E}_{\Pi}\Big[\sum_{t=1}^{n} r^*(t) - r(t)\Big] \\ &= \mathbb{E}\Big[\sum_{t=1}^{n} r(\mathcal{G}_t,\mathcal{H}_t) - r(\mathcal{A}_t,\mathcal{H}_t)\Big] \\ &= \mathbb{E}\Big[\sum_{t=1}^{n} \Delta_t \cdot \mathbb{1}\Big\{(r(\mathcal{G}_t,\mathcal{H}_t) \neq r(\mathcal{A}_t,\mathcal{H}_t))\Big\}\Big] \\ &\leq \Delta_{max} \cdot \mathbb{E}\Big[\sum_{\mathcal{S}\in H(\mathbb{K})} N_{\mathcal{S}}(n)\Big], \end{aligned} \tag{21}$$

where Δ_t is the difference between the reward achieved by the algorithm to the highest reward that could have been achieved at iteration t (i.e., $\Delta_t = r(\mathcal{G}_t,\mathcal{H}_t) - r(\mathcal{A}_t,\mathcal{H}_t) \geq 0$), and $N_{\mathcal{S}}(n)$ is the number of iterations up until the nth iteration in which selection $\mathcal{S} \in H(\mathbb{K})$ was selected and the reward given from it was strictly less than the reward that would have been received by genie's selection in the same iteration. In addition, we define a K-dimensional counter vector $\widetilde{\mathbf{N}}(n) = (\widetilde{\mathbf{N}}_1(n), \widetilde{\mathbf{N}}_2(n), ..., \widetilde{\mathbf{N}}_K(n))$, corresponding to the K clients as follows. For each iteration, in which the selection $\mathcal{A}_t \in H(\mathbb{A}_t)$ achieves a lower reward than the reward of genie's selection, i.e., $r(\mathcal{A}_t,\mathcal{H}_t) < r(\mathcal{G}_t,\mathcal{H}_t)$, then the counter of the client that has been selected the fewest number of times up to this iteration among all the selected clients that were selected in this iteration is incremented by 1. Formally, for each client (say i) let $\mathcal{T}_i(n)$ denote the set of time indices up to time n that satisfy the following conditions: (i) Client i was selected, i.e., $i \in \mathcal{A}_t$ for all $t \in \mathcal{T}_i(n)$; (ii) the counter $c_{i,t}$ of client i is the minimal among all selected clients, i.e., $i = \arg\min_{k\in\mathcal{A}_t} c_{k,t}$, for all $t \in \mathcal{T}_i(n)$; and (iii) the selection $\mathcal{A}_t \in H(\mathbb{A}_t)$ achieves a lower reward than genie's selection, i.e., $r(\mathcal{A}_t,\mathcal{H}_t) < r(\mathcal{G}_t,\mathcal{H}_t)$ for all $t \in \mathcal{T}_i(n)$. Then,

$$\widetilde{\mathbf{N}}_i(n) = |\mathcal{T}_i(n)|. \tag{22}$$

Next, we aim at upper bounding $N_{\mathcal{S}}(n)$ for each $\mathcal{S} \in H(\mathbb{K})$. Note that based on the definition of $\widetilde{\mathbf{N}}(n)$, for every iteration in which the selection $\mathcal{A}_t$ has a lower reward than genie's selection, one of the coordinates in the vector $\widetilde{\mathbf{N}}(n)$ is incremented by 1. Therefore,

$$\sum_{\mathcal{S}\in H(\mathbb{K})} N_{\mathcal{S}}(n) = \sum_{k=1}^{K} \widetilde{\mathbf{N}}_k(n), \tag{23}$$

which implies

$$\mathbb{E}\Big[\sum_{\mathcal{S}\in H(\mathbb{K})} N_{\mathcal{S}}(n)\Big] = \sum_{k=1}^{K} \mathbb{E}\Big[\widetilde{\mathbf{N}}_k(n)\Big]. \tag{24}$$

Let $I_k(t)$ be the indicator for the event that $\widetilde{\mathbf{N}}_k(t)$ is incremented by 1 at iteration t. Hence, we obtain

$$\begin{aligned} \widetilde{\mathbf{N}}_i(n) &= \sum_{t=1}^{n} \mathbb{1}\Big\{(I_i(t)=1)\Big\} \\ &\leq 1 + \sum_{t=\lceil \frac{\mathbb{K}}{m}\rceil+1}^{n} \mathbb{1}\Big\{(I_i(t)=1)\Big\} \\ &\leq l + \sum_{t=\lceil \frac{\mathbb{K}}{m}\rceil+1}^{n} \mathbb{1}\Big\{(I_i(t)=1, \widetilde{\mathbf{N}}_i(t) \geq l)\Big\} \\ &\leq l + \sum_{t=\lceil \frac{\mathbb{K}}{m}\rceil+1}^{n} \mathbb{1}\Big\{(\min_{k\in\mathcal{G}_t} \mathrm{ucb}(k,t) + \frac{\alpha}{m}\sum_{k\in\mathcal{G}_t} g_{k,\Theta}(\mathcal{H}_t) < \\ &\min_{k\in\mathcal{A}_t} \mathrm{ucb}(k,t) + \frac{\alpha}{m}\sum_{k\in\mathcal{A}_t} g_{k,\Theta}(\mathcal{H}_t), \widetilde{\mathbf{N}}_i(t) \geq l)\Big\}, \end{aligned} \tag{25}$$

where the last inequality follows since the algorithm chooses action $\mathcal{A}_t \neq \mathcal{G}_t$ that solves (13) although the

reward is maximized by $\mathcal{G}_t$. Note that according to the definition of $\widetilde{\mathbf{N}}$, for all $k \in \mathcal{A}_t$ we have: $\widetilde{\mathbf{N}}_i(t) \leq c_{k,t}$. Therefore, since in the indicator function there is an intersection with the event that $\widetilde{\mathbf{N}}_i(t) \geq l$ we have for all $k \in \mathcal{A}_t$ that: $l \leq \widetilde{\mathbf{N}}_i(t) \leq c_{k,t}$ in (25). Denote $h_{c_{k,t}} = \sqrt{\frac{(m+1)\ln t}{c_{k,t}}}$, and $\overline{\mu_{c_k}}$ denotes the sampled mean of the $\frac{\tau_{min}}{\tau_k}$ of client k after c_k observations. Using these notations, we can upper bound $\widetilde{\mathbf{N}}_i(n)$ by:

$$
\begin{aligned}
&\widetilde{\mathbf{N}}_i(n) \\
&\leq l + \sum_{t=\lceil \frac{\mathbb{K}}{m} \rceil + 1}^{n} \mathbb{1}\Bigg\{ \min_{k \in \mathcal{G}_t} \{\overline{\mu_{k,t}} + h_{c_{k,t}}\} + \frac{\alpha}{m} \sum_{k \in \mathcal{G}_t} g_{k,\Theta}(\mathcal{H}_t) < \\
&\qquad \min_{k \in \mathcal{A}_t} \{\overline{\mu_{k,t}} + h_{c_{k,t}}\} + \frac{\alpha}{m} \sum_{k \in \mathcal{A}_t} g_{k,\Theta}(\mathcal{H}_t), \widetilde{\mathbf{N}}_i(t) \geq l \Bigg\} \\
&\leq l + \sum_{t=\lceil \frac{\mathbb{K}}{m} \rceil + 1}^{n} \mathbb{1}\Bigg\{ \min_{0 \leq c_{\hat{k}_{1,t}}, c_{\hat{k}_{2,t}}, \ldots, c_{\hat{k}_{m,t}} \leq t} \\
&\Bigg\{ \min_{j \in \{1,\ldots,m\}} \{\overline{\mu_{c_{\hat{k}_{j,t}}}} + h_{c_{\hat{k}_{j,t}}}\} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) \Bigg\} < \\
&\max_{l \leq c_{\tilde{k}_{1,t}}, c_{\tilde{k}_{2,t}}, \ldots, c_{\tilde{k}_{m,t}} \leq t} \Bigg\{ \min_{j \in \{1,\ldots,m\}} \{\overline{\mu_{c_{\tilde{k}_{j,t}}}} + h_{c_{\tilde{k}_{j,t}}}\} + \\
&\qquad \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t) \Bigg\} \Bigg\} \\
&\leq l + \sum_{t=\lceil \frac{\mathbb{K}}{m} \rceil + 1}^{n} \sum_{c_{\hat{k}_{1,t}}=1}^{t} \cdots \sum_{c_{\hat{k}_{m,t}}=1}^{t} \sum_{c_{\tilde{k}_{1,t}}=l}^{t} \cdots \sum_{c_{\tilde{k}_{m,t}}=l}^{t} \cdot \\
&\mathbb{1}\Bigg\{ \min_{j \in \{1,\ldots,m\}} \{\overline{\mu_{c_{\hat{k}_{j,t}}}} + h_{c_{\hat{k}_{j,t}}}\} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) < \\
&\qquad \min_{j \in \{1,\ldots,m\}} \{\overline{\mu_{c_{\tilde{k}_{j,t}}}} + h_{c_{\tilde{k}_{j,t}}}\} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t) \Bigg\},
\end{aligned}
\tag{26}
$$

where $\{\hat{k}_{j,t} : 1 \leq j \leq m\}$ and $\{\tilde{k}_{j,t} : 1 \leq j \leq m\}$ are the clients in genie's selection and the server's selection at iteration t, respectively. Let $\hat{k}'_t$, $\tilde{k}'_t$, respectively, be the clients in genie's selection and the server's selection at iteration t that minimizes the expression in the upper bound we derived, i.e.,

$$
\hat{k}'_t = \underset{\hat{k}_j \in \{\hat{k}_1, \ldots, \hat{k}_m\}}{\arg\min} \{\overline{\mu_{c_{\hat{k}_{j,t}}}} + h_{c_{\hat{k}_{j,t}}}\}, \tag{27}
$$

$$
\tilde{k}'_t = \underset{\tilde{k}_j \in \{\tilde{k}_1, \ldots, \tilde{k}_m\}}{\arg\min} \{\overline{\mu_{c_{\tilde{k}_{j,t}}}} + h_{c_{\tilde{k}_{j,t}}}\}. \tag{28}
$$

Now, we claim that the event

$$
\Bigg\{ \overline{\mu_{\hat{k}'_t}} + h_{c_{\hat{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) < \overline{\mu_{\tilde{k}'_t}} + h_{c_{\tilde{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t) \Bigg\}
$$

implies that at least one of the 3 following events must occur:

(i) $\overline{\mu_{\hat{k}'_t}} + h_{c_{\hat{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) \leq \mu_{\hat{k}'_t} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t)$;

(ii) $\overline{\mu_{\tilde{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t) \geq \mu_{\tilde{k}'_t} + h_{c_{\tilde{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t)$;

(iii) $\mu_{\hat{k}'_t} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) < \mu_{\tilde{k}'_t} + 2h_{c_{\tilde{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t)$.

We next prove this claim by contradiction. Assume that all three inequalities do not hold. Therefore, it follows that:

$$
\begin{aligned}
\overline{\mu_{\hat{k}'_t}} + h_{c_{\hat{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) &> \mu_{\hat{k}'_t} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) \\
&\geq \mu_{\tilde{k}'_t} + 2h_{c_{\tilde{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t) \\
&> \overline{\mu_{\tilde{k}'_t}} + h_{c_{\tilde{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t),
\end{aligned}
\tag{29}
$$

where the first transition is derived from (i), the second from (iii), the last from (ii), and all three together contradict the event. Now, we aim at upper bounding the probabilities Pr(i), Pr(ii) that events (i) and (ii) will occur:

$$
\begin{aligned}
Pr(\text{i}) &= Pr\Big(\overline{\mu_{\hat{k}'_t}} + h_{c_{\hat{k}'_t}} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) \\
&\qquad \leq \mu_{\hat{k}'_t} + \frac{\alpha}{m} \sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) \Big) \\
&= Pr\big(\overline{\mu_{\hat{k}'_t}} + h_{c_{\hat{k}'_t}} \leq \mu_{\hat{k}'_t} \big) \\
&\leq e^{-2c_{\hat{k}'_t}^2 \cdot \frac{(m+1)\ln t}{c_{\hat{k}'_t}} \cdot \frac{1}{c_{\hat{k}'_t}}} = t^{-2(m+1)},
\end{aligned}
\tag{30}
$$

where the inequality is due to Hoeffding's inequality. Similarly, we can upper bound Pr(ii) by the same upper bound and obtain:

$$
Pr(\text{ii}) \leq t^{-2(m+1)}. \tag{31}
$$

To ensure that (iii) will not occur we need to put a lower bound on l (i.e., the minimum number of times a client should be selected when he has the minimum number of selections so far among the clients in the current selection):

$$
\begin{aligned}
&\big\{ \mu_{\hat{k}'_t} + \tfrac{\alpha}{m} \textstyle\sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t) < \\
&\qquad \mu_{\tilde{k}'_t} + 2h_{c_{\tilde{k}'_t}} + \tfrac{\alpha}{m} \textstyle\sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t) \big\} \Leftrightarrow \\
&\big\{ \mu_{\hat{k}'_t} + \tfrac{\alpha}{m} \textstyle\sum_{j=1}^{m} g_{\hat{k}_{j,t},\Theta}(\mathcal{H}_t)
\end{aligned}
$$

$$-\mu_{\tilde{k}'_t} - \frac{\alpha}{m}\sum_{j=1}^{m} g_{\tilde{k}_{j,t},\Theta}(\mathcal{H}_t) > 2h_{c_{\tilde{k}'_t}}\}.$$

Denote the LHS of the last inequality by $\Delta_{\mathcal{G}_t,\mathcal{A}_t,\alpha}$. Then, for the last inequality to hold, we can demand that for every $\mathcal{A}_t$ and $\mathcal{G}_t$ selections by the algorithm and genie, respectively (which satisfy $r(\mathcal{A}_t,\mathcal{H}_t) < r(\mathcal{G}_t,\mathcal{H}_t)$):

$$\Delta_{\mathcal{G}_t,\mathcal{A}_t,\alpha} \geq 2\sqrt{\frac{(m+1)\ln t}{c_{\tilde{k}'_t}}}, \tag{32}$$

and because we have already shown that $\forall k \in \mathcal{A}_t : l \leq c_{k,t}$ and $t \leq n$, it is sufficient to demand

$$\Delta_{\mathcal{G}_t,\mathcal{A}_t,\alpha} \geq 2\sqrt{\frac{(m+1)\ln n}{l}}. \tag{33}$$

Therefore, for $l \geq \frac{4(m+1)\ln n}{\Delta^2_{\mathcal{G}_t,\mathcal{A}_t,\alpha}}$ for every t, or alternatively, we can choose $l = \left\lceil \frac{4(m+1)\ln n}{\Delta^2_{min}} \right\rceil$ and obtain that inequality (iii) will not be met. Hence, only one of the first two inequalities must occur, and we obtain

$$\begin{aligned}
&\mathbb{E}\left[\widetilde{\mathbf{N}}_i(n)\right] \\
&\leq l + \sum_{t=\lceil \frac{\mathbb{K}}{m} \rceil+1}^{n} \sum_{c_{\tilde{k}_{1,t}}=1}^{t} \cdots \sum_{c_{\tilde{k}_{m,t}}=1}^{t} \sum_{c_{\tilde{k}_{1,t}}=l}^{t} \cdots \sum_{c_{\tilde{k}_{m,t}}=l}^{t} \\
&\qquad\qquad (Pr(i) + Pr(ii)) \\
&\leq \left\lceil \frac{4(m+1)\ln n}{\Delta^2_{min}} \right\rceil \\
&+ \sum_{t=\lceil \frac{\mathbb{K}}{m} \rceil+1}^{\infty} \sum_{c_{\tilde{k}_{1,t}}=1}^{t} \cdots \sum_{c_{\tilde{k}_{m,t}}=1}^{t} \sum_{c_{\tilde{k}_{1,t}}=1}^{t} \cdots \sum_{c_{\tilde{k}_{m,t}}=1}^{t} \\
&\qquad\qquad 2t^{-2(m+1)} \\
&\leq \frac{4(m+1)\ln t}{\Delta^2_{min}} + 1 + \frac{\pi^2}{3}.
\end{aligned} \tag{34}$$

Finally, we can upper bound the regret by:

$$R(n) \leq \Delta_{max} K \cdot \left(\frac{4(m+1)\ln n}{\Delta^2_{min}} + 1 + \frac{\pi^2}{3} \right).$$

□

Appendix B
Proof of Theorem 2

In this appendix we provide the proof for Theorem 2. From [46], the following conditions are sufficient to guarantee the convergence of cooling procedure to the state with the lowest energy (or, alternatively, the highest energy, depending on the problem formulation):

(i) Weak Reversibility: For any energy E and any two states s_1 and s_2, s_1 is reachable at height energy E from state s_2, i.e., there exists a path from s_1 to s_2 that goes only through states with energy E or higher) iff s_2 is reachable from s_1 at height E.

(ii) The temperature is set to $T_i = \frac{d}{\log(i+1)}$, where d is greater then the difference between the energies of the highest local maxima and the minimum energy state.

(iii) The Markov chain is irreducible.

Next, we prove that all conditions are met by ALSA. Condition (i) follows immediately by the definition of the neighborhoods which is symmetrical, i.e., in a graph with symmetric neighborhoods, any path from one node to another can also be in the opposite direction and go through the exact same states. Regarding condition (ii), since Δ_{max} is defined as the largest possible energy difference between any two states, it is in particular greater than the difference in energies between any local maximum and the state with the minimum energy. Therefore, condition (ii) also holds.

Finally, we need to show that condition (iii) holds. We will show this by proving that in the newly structured state graph by ALSA, from every possible state there exists a path that reaches s^*, and due to the symmetric neighborhoods, this will complete the proof. Denote s_0 as some arbitrary state (selection). Define the state s_1 to be $s_1 = \big(s_0 \backslash \{\arg\min_{k\in\mathcal{S}_0} \text{ucb}(k,t)\}\big) \cup \{\arg\min_{k\in\mathcal{S}^*} \text{ucb}(k,t)\}$. Note that s_1 and s_0 are neighboring states. For $j = 1, 2, 3, ...$ let us define the rest of the path with two phases (P1,P2) as follows:

(P1) As long as

$$\arg\min_{k\in\mathcal{S}^*} \text{ucb}(k,t) \neq \arg\min_{k\in\mathcal{S}_j} \text{ucb}(k,t):$$
$$\text{define } s_{j+1} = \big(s_j \backslash \{\arg\min_{k\in\mathcal{S}_j} \text{ucb}(k,t)\}\big) \cup \big\{\arg\max_{k\in\mathcal{S}^*} \text{ucb}(k,t)\big\}.$$

(P2) After Phase 1 ends, as long as $s^* \neq s_j$:

$$\text{define } s_{j+1} = \big(s_j \backslash \{\arg\min_{k\in\mathcal{S}_0} g_{k,\Theta}(\mathcal{H}_t)\}\big) \cup \{\arg\max_{k\in\mathcal{S}^*} g_{k,\Theta}(\mathcal{H}_t)\}.$$

Note that each one of the phases lasts a finite amount of iterations, i.e., $\exists n \in \mathbb{N} : s_n = s^*$. Note that for every $j \in \mathbb{N}$ the state s_j and s_{j+1} are neighbors, which implies that $P = (s_0, s_1, ..., s_n = s^*)$ defined by the last phases is a path from s_0 to s^*, which completes the proof. □

References

[1] D. Ban Ami, K. Cohen, and Q. Zhao, "Client selection for generalization in accelerated federated learning: A bandit approach." To appear in the Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), June 2023.

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*, pp. 1273–1282, PMLR, 2017.

[3] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečnỳ, S. Mazzocchi, B. McMahan, *et al.*, "Towards federated learning at scale: System design," *Proceedings of Machine Learning and Systems*, vol. 1, pp. 374–388, 2019.

[4] T. Gafni, N. Shlezinger, K. Cohen, Y. C. Eldar, and H. V. Poor, "Federated learning: A signal processing perspective," *IEEE Signal Processing Magazine*, vol. 39, no. 3, pp. 14–41, 2022.

[5] R. Srikant and L. Ying, *Communication networks: an optimization, control, and stochastic networks perspective*. Cambridge University Press, 2013.

[6] M. M. Amiri and D. Gündüz, "Federated learning over wireless fading channels," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3546–3557, 2020.

[7] M. M. Amiri and D. Gündüz, "Machine learning at the wireless edge: Distributed stochastic gradient descent over-the-air," *IEEE Transactions on Signal Processing*, vol. 68, pp. 2155–2169, 2020.

[8] T. Sery and K. Cohen, "A sequential gradient-based multiple access for distributed learning over fading channels," in *57th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pp. 303–307, IEEE, 2019.

[9] T. Sery and K. Cohen, "On analog gradient descent learning over multiple access fading channels," *IEEE Transactions on Signal Processing*, vol. 68, pp. 2897–2911, 2020.

[10] T. Sery, N. Shlezinger, K. Cohen, and Y. C. Eldar, "Over-the-air federated learning from heterogeneous data," *IEEE Transactions on Signal Processing*, vol. 69, pp. 3796–3811, 2021.

[11] R. Paul, Y. Friedman, and K. Cohen, "Accelerated gradient descent learning over multiple access fading channels," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 2, pp. 532–547, 2021.

[12] J. Konečnỳ, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.

[13] C. Tekin and M. Liu, "Online learning of rested and restless bandits," *IEEE Transactions on Information Theory*, vol. 58, no. 8, pp. 5588–5611, 2012.

[14] H. Liu, K. Liu, and Q. Zhao, "Learning in a changing world: Restless multiarmed bandit with unknown dynamics," *IEEE Transactions on Information Theory*, vol. 59, no. 3, pp. 1902–1916, 2012.

[15] O. Naparstek and K. Cohen, "Deep multi-user reinforcement learning for distributed dynamic spectrum access," *IEEE transactions on wireless communications*, vol. 18, no. 1, pp. 310–323, 2018.

[16] I. Bistritz and A. Leshem, "Distributed multi-player banditsa game of thrones approach," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[17] T. Gafni and K. Cohen, "Learning in restless multiarmed bandits via adaptive arm sequencing rules," *IEEE Transactions on Automatic Control*, vol. 66, no. 10, pp. 5029–5036, 2020.

[18] T. Gafni and K. Cohen, "Distributed learning over markovian fading channels for stable spectrum access," *IEEE Access*, vol. 10, pp. 46652–46669, 2022.

[19] T. Gafni, M. Yemini, and K. Cohen, "Learning in restless bandits under exogenous global markov process," *IEEE Transactions on Signal Processing, doi: 10.1109/TSP.2022.3224790*, pp. 1–16, 2022.

[20] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, *et al.*, "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[21] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[22] T. Li, M. Sanjabi, A. Beirami, and V. Smith, "Fair resource allocation in federated learning," *arXiv preprint arXiv:1905.10497*, 2019.

[23] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *ICC 2019-2019 IEEE international conference on communications (ICC)*, pp. 1–7, IEEE, 2019.

[24] J. Xu and H. Wang, "Client selection and bandwidth allocation in wireless federated learning networks: A long-term perspective," *IEEE Transactions on Wireless Communications*, vol. 20, no. 2, pp. 1188–1200, 2020.

[25] S. AbdulRahman, H. Tout, A. Mourad, and C. Talhi, "Fedmccs: Multicriteria client selection model for optimal iot federated learning," *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4723–4735, 2020.

[26] E. Rizk, S. Vlaski, and A. H. Sayed, "Federated learning under importance sampling," *IEEE Transactions on Signal Processing*, vol. 70, pp. 5381–5396, 2022.

[27] H. H. Yang, A. Arafa, T. Q. Quek, and H. V. Poor, "Age-based scheduling policy for federated learning in mobile edge networks," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 8743–8747, 2020.

[28] M. E. Ozfatura, J. Zhao, and D. Gündüz, "Fast federated edge learning with overlapped communication and computation and channel-aware fair client scheduling," in *2021 IEEE 22nd International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, pp. 311–315, IEEE, 2021.

[29] W. Xia, T. Q. Quek, K. Guo, W. Wen, H. H. Yang, and H. Zhu, "Multi-armed bandit-based client scheduling for federated learning," *IEEE Transactions on Wireless Communications*, vol. 19, no. 11, pp. 7108–7123, 2020.

[30] N. Yoshida, T. Nishio, M. Morikura, and K. Yamamoto, "Mab-based client selection for federated learning with uncertain resources in mobile networks," in *2020 IEEE Globecom Workshops (GC Wkshps*, pp. 1–6, IEEE, 2020.

[31] Y. J. Cho, S. Gupta, G. Joshi, and O. Yağan, "Bandit-based communication-efficient client selection strategies for federated learning," in *2020 54th Asilomar Conference on Signals, Systems, and Computers*, pp. 1066–1069, IEEE, 2020.

[32] B. Xu, W. Xia, J. Zhang, T. Q. Quek, and H. Zhu, "Online client scheduling for fast federated learning," *IEEE Wireless Communications Letters*, vol. 10, no. 7, pp. 1434–1438, 2021.

[33] T. Huang, W. Lin, W. Wu, L. He, K. Li, and A. Y. Zomaya, "An efficiency-boosting client selection scheme for federated learning with fairness guarantee," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1552–1564, 2020.

[34] Q. Zhao, "Multi-armed bandits: Theory and applications to online learning in networks," *Synthesis Lectures on Communication Networks*, vol. 12, no. 1, pp. 1–165, 2019.

[35] N. Levine, K. Crammer, and S. Mannor, "Rotting bandits," *Advances in neural information processing systems*, vol. 30, 2017.

[36] W. Chen, Y. Wang, and Y. Yuan, "Combinatorial multi-armed bandit: General framework and applications," in *International conference on machine learning*, pp. 151–159, PMLR, 2013.

[37] M. Hardt, E. Price, and N. Srebro, "Equality of opportunity in supervised learning," *Advances in neural information processing systems*, vol. 29, 2016.

[38] M. Mohri, G. Sivek, and A. T. Suresh, "Agnostic federated learning," in *International Conference on Machine Learning*, pp. 4615–4625, PMLR, 2019.

[39] Y. Shi, H. Yu, and C. Leung, "A survey of fairness-aware federated learning," *arXiv preprint arXiv:2111.01872*, 2021.

[40] H. Xu and S. Mannor, "Robustness and generalization," *Machine learning*, vol. 86, no. 3, pp. 391–423, 2012.

[41] X. Zhu and X. Wu, "Class noise vs. attribute noise: A quantitative study," *Artificial intelligence review*, vol. 22, no. 3, pp. 177–210, 2004.

[42] A. Atla, R. Tada, V. Sheng, and N. Singireddy, "Sensitivity of different machine learning algorithms to noise," *Journal of Computing Sciences in Colleges*, vol. 26, no. 5, pp. 96–103, 2011.

[43] D. F. Nettleton, A. Orriols-Puig, and A. Fornells, "A study of the effect of different types of noise on the precision of supervised learning techniques," *Artificial intelligence review*, vol. 33, no. 4, pp. 275–306, 2010.

[44] S. Gupta and A. Gupta, "Dealing with noise problem in machine learning data-sets: A systematic review," *Procedia Computer Science*, vol. 161, pp. 466–474, 2019.

[45] Y. J. Cho, J. Wang, and G. Joshi, "Towards understanding biased client selection in federated learning," in *International Conference on Artificial Intelligence and Statistics*, pp. 10351–10375, PMLR, 2022.

[46] B. Hajek, "Cooling schedules for optimal annealing," *Mathematics of operations research*, vol. 13, no. 2, pp. 311–329, 1988.